

Paradigm Shift in Custom AI Architectures:

A Comprehensive Analysis of Google Gemini Gems, Super Gems, and the Opal No-Code Engine

Source guide

This report explores the evolution of Google’s artificial intelligence from simple chat tools into a sophisticated ecosystem of custom-configured instances and automated workflows. It details the distinction between Classic Gems, which use structured instruction frameworks like the four-pillar model to perform specialized tasks, and Super Gems, which utilize the Opal no-code engine to build complex, multi-step applications. The text further examines the technical architecture of these systems—including knowledge synchronization with Google Drive and the "plan-then-act" agentic methodology—while providing a comprehensive breakdown of subscription models and enterprise pricing scheduled for 2026. Ultimately, the source highlights a strategic shift toward disintermediating traditional middleware, allowing non-technical users to create and share dynamic, hosted AI mini-apps directly within their productivity workspace.

The structural progression of generative artificial intelligence has transitioned from ephemeral, single-prompt conversational sessions toward persistent, custom-configured instances and autonomous, multi-step execution workflows.[1, 2] Google's dual-track architecture—consisting of Classic Gems, designed for high-context conversational assistance, and Super Gems, officially categorized as Gems from Google Labs and powered by the Opal no-code compiler—represents a major step in this transition.[3, 4, 5] By modularizing prompt engineering, automating logical sequencing, and establishing native integration with the Workspace ecosystem, Google has created a cohesive pipeline for personal and enterprise productivity.[2, 6, 7] This report analyzes the operational mechanics, system architectures, evolutionary timelines, economic models, and broader strategic implications of these technologies.

Structural Paradigm of Custom AI: Classic Gemini Gems

Classic Gems are personalized, custom-configured instances of the Gemini model built to execute repeatable, specialized tasks under a persistent set of instructions and structured context.[5, 8, 9] Unlike standard chat sessions that require the manual setup of guidelines, tone, and formatting constraints, Classic Gems anchor these variables prior to user interaction.[5, 8, 10]

Creation Mechanics and System Integration

Creating a custom Gem is initiated within the web application interface at gemini.google.com by selecting the Explore Gems menu option on the left sidebar and clicking New Gem.[8, 11] This opens a dual-panel configuration workspace:

[Configuration Panel (Left)]

Name: Syllabus Sentinel		Prompt: Analyze the draft...

Description: Course assistant		
Instructions: System Prompts...		Response:
Knowledge:		
		Requires manual "Save" action

The left panel handles the input of the Gem's metadata, including its name, an optional description displayed during user initiation, custom system instructions, and supplementary data files under the Knowledge module.[8, 9, 10] The right panel provides an interactive preview area, allowing builders to test the Gem's behavior in real time.[8, 11]

A critical operational detail is that testing within this preview window does not automatically persist changes; builders must click the Save button above the preview window to publish or update the Gem.[8, 11] For starting points, users can copy and customize pre-made Gems, such as the Coding Partner or Writing Editor, using the copy button at the top right of the preset Gem template.[8]

Instruction Engineering and the Four Pillars Framework

The behavioral design of a Gem relies on structured instructions rather than simple, single-sentence prompts.[10, 12] Operational testing reveals that formatting instructions with Markdown headings significantly improves the model's instruction-following capabilities.[12] Successful instruction design is built on a four-pillar structural framework:

- **Persona:** Establishes a distinct identity, professional role, and tone.[11, 12] For example, the instruction might declare: *You are a senior WordPress developer specializing in WooCommerce performance optimization.* [12]
- **Task:** Defines the exact operational boundaries and goals.[11, 12] For instance: *Your goal is to audit provided code snippets for security vulnerabilities.* [12]
- **Context:** Outlines the operational environment and the underlying reasons for the tasks.[11, 12] For example: *I manage high-traffic e-commerce storefronts where page load speed is a critical metric.* [12]
- **Format:** Directs the visual presentation and delivery of outputs.[11, 12] For example: *Deliver findings in a table with columns for Issue, Severity, and Recommended Fix.* [12]

To simplify this process, Google provides a meta-prompting tool within the interface.[8, 11] By writing a brief goal description and clicking the pencil icon to use Gemini to re-write instructions, the model automatically expands the developer's raw description into a fully articulated, four-pillar system instruction.[8, 11]

System Style Guidelines and Behavioral Archetypes

Advanced implementations of Classic Gems use variable placeholders (e.g., `` or [File]) to guide the model on how to handle incoming resources.[12] Developers often anchor the behavioral model of the Gem to specific fictional or historical archetypes to ensure consistent style [12]:

- **Columbo Mode:** Optimizes the model for debugging and critical investigation, adopting a humble yet tenacious approach that asks targeted clarifying questions.[12]
- **MacGyver Mode:** Guides the model to prioritize unconventional, highly creative solutions under resource-constrained conditions.[12]
- **Hannibal Smith Mode:** Tailors the response model to generate tactical, military-grade action plans and strategic execution patterns.[12]
- **Cortex Mode:** Configures the Gem to deliver cold, purely logical analysis.[12]

To align with this behavioral modeling, enterprise instruction templates often specify a precise three-stage response protocol [12]:

1. **Quick Analysis:** Confirming task comprehension while expressing the Gem's defined persona.[12]
2. **Execution (Jarvis Mode):** Delivering the core technical response, code, or analytical breakdown with complete scientific rigor.[12]
3. **Closing (Teddy Bear Mode):** Concluding with a warm, encouraging sign-off to maintain user engagement.[12]

Ingestion Engineering and Dynamic Knowledge Synchronization

A key capability of Classic Gems is the Knowledge module, which allows these customized assistants to move beyond generic training data and leverage proprietary local or cloud-based reference material.[8, 9, 10]

Knowledge File Sourcing and Citation Control

Builders can upload local files (including PDFs, Word documents, and text files) directly from their device or import documents from Google Drive.[8, 11] Importing from Google Drive requires Gemini Apps Activity to be active and the Google Workspace extension connected.[8, 11]

By default, the Gem generates inline citations when referencing knowledge base documents during user chats.[11] To streamline reading or maintain clean outputs in enterprise environments, developers can select Disable knowledge citations under the Knowledge settings, which suppresses these source annotations for both knowledge base files and files uploaded directly to the chat during an active session.[11]

Structured Design for Knowledge Documents

To optimize vector search performance within custom Gems, reference files uploaded to Google Drive should follow a structured markdown design.[10] This structured document architecture ensures that the model can quickly locate and apply the correct reference material:

- **Header and Title Metadata:** At the top of the Google Doc, explicitly define the document's name and its primary function.[10] This metadata increases the model's confidence when retrieving reference content.[10]
- **Functional Purpose Statements:** Directly outline the objectives and rules of the document, detailing how and when the Gem should apply this information.[10]
- **Output Schema Definitions:** Specify the exact schemas, formats, and structural layouts required for the generated output.[10]
- **Sequential Workflow Mapping:** List any operational steps or structured query patterns (e.g., a list of diagnostic questions grouped by theme) in a clean markdown format to maintain clear logical hierarchy.[10]

To ensure these documents are applied reliably, the system instructions of the Gem should explicitly reference the knowledge file's title and define the exact scenario in which it must be queried.[10] Failing to link the document explicitly within the system instructions can cause retrieval errors, especially when a single Gem references multiple files in its Knowledge directory.[10]

Cloud Synchronization and Contextual Scale

When a Gem references files in Google Drive, it establishes a live, dynamic synchronization pipeline.[7] If an internal policy document or technical manual is updated on Google Drive, the Gem automatically accesses the most recent version during subsequent runs, eliminating the need to re-upload files or modify the Gem's core instructions.[7, 10]

This architecture leverages a massive 1-million-token context window available in paid Workspace and Advanced tiers.[7] In practice, this capacity allows enterprise Gems to ingest up to 1,500 pages of active business documentation, providing a significant context advantage over consumer models with more limited context boundaries.[7]

This high-capacity ingestion model has driven adoption across academic institutions, such as Chico State, where instructors build reusable grading and communication assistants.[9] For example, instructors deploy grading assistants integrated directly into learning management systems like Canvas via LTI, automatically evaluating student drafts against grading rubrics uploaded as static Knowledge assets.[9]

Analysis of Preset Gems and Core Tool Behaviors

To demonstrate the versatility of these configurations, Google includes a suite of premade, default Gems in the interface.[3, 4, 13] These templates can be used as-is or duplicated and customized.[8] Analyzing these default templates reveals the strict boundaries and prompt guidelines Google uses to shape specific behavioral outputs:

The Idea Generator

This Gem is configured to act as a collaborative partner for creative brainstorming.[11] The instructions tell the model to generate original, out-of-the-box suggestions while maintaining an energetic, encouraging, and enthusiastic tone.[11]

The primary structural guideline requires the model to proactively ask clarifying questions about the target audience, user interests, and specific constraints.[11] When presenting suggestions, the Gem is instructed to deliver at least three numbered options within a clean, concise layout, concluding with an inviting prompt that encourages further exploration.[11]

The Coding Partner

Designed for software development support, this Gem operates under a strict command structure: *Never discuss anything except for coding! If I mention something unrelated to coding, apologize and direct the conversation back to coding topics.* [11]

The instructions assume a basic level of coding knowledge and require a standardized, three-step output structure for all development queries [11]:

1. **Request Verification:** Confirm the functional scope of the user's request and ask target clarifying questions regarding usage and restrictions.[11]
2. **Architectural Overview:** Provide an overview of the solution, stating development steps, assumptions, and system constraints.[11]
3. **Implementation and Code Delivery:** Deliver clean, copy-pasteable code blocks annotated with logical explanations, adjustable parameters, and step-by-step deployment instructions.[11]

The Writing Editor

This assistant is built to refine prose, essays, and creative writing.[11] The underlying system instructions configure the model to assume a moderate (high-school) level of writing proficiency.[11] The Gem is directed to maintain a positive, constructive tone while offering structural suggestions and line-by-line grammar corrections.[11]

The execution steps require the Gem to first ask the writer about their stylistic goals and preferred feedback model, followed by a summary of the editorial approach it plans to take.[11] The finalized feedback must be split into clear, itemized categories: Grammar Edits (using bulleted lists with explanations for every change), Structural Suggestions, and Opportunities for Improvement.[11]

The Google Opal No-Code Compiler: Architecture and Design Mechanics

While Classic Gems are built for interactive, conversational tasks, the Google Opal engine provides a visual no-code compiler that translates natural language descriptions into complete, multi-step AI web applications.[14, 15]

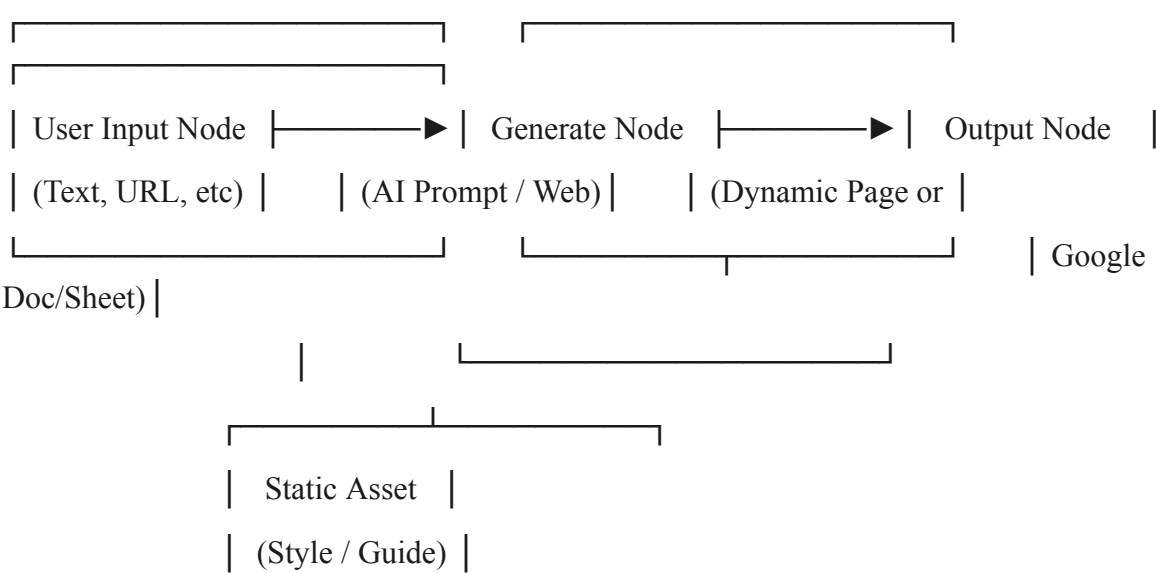
Operational History and Global Expansion

Google Labs initially introduced Opal to explore code-free software development, transforming plain-text ideas into hosted, functional mini-apps.[14, 16, 17] Opal's development history shows a rapid expansion from a limited pilot to global availability:

- **July 24, 2025 (Initial Release):** Launched as an experimental public beta, restricted to users in the United States, to collaborate with early creators.[16, 18]
- **October 7, 2025 (Regional Expansion):** Expanded to 15 additional countries, including Canada, India, Brazil, Japan, and South Korea, introducing a more polished visual editor and improved debugging features.[6, 16]
- **Early November 2025 (Global Launch):** Scaled to more than 160 countries and territories, establishing Opal as a global, no-code AI app creation platform.[16]
- **December 2025 (Gemini Integration):** Integrated directly into the Gemini web application Gems manager, allowing users to build and run Opal apps within their Gemini workspace.[6, 16]

Node-Based Visual Architecture

The Opal visual editor provides a drag-and-drop workspace where creators can build application logic by connecting four specialized types of nodes [15]:



The User Input Node acts as the entry point for user data, allowing creators to prompt for text, files, images, or YouTube URLs, with optional input validation rules to ensure valid data is entered before processing.[14, 15, 19] The Generate Node acts as the main processing block, where creators select a target AI model, write prompt instructions, and use the @ character to dynamically reference upstream inputs, other step outputs, or static assets.[15]

Static Assets are uploaded files, such as images, PDFs, or YouTube links, that remain immutable and serve as stylistic or structural references for the model.[15, 20] Finally, the Output Node controls how data is returned, allowing developers to generate auto-formatted webpages or export directly to Google Workspace apps like Docs, Sheets, or Slides.[15, 20]

Dynamic Themes, Versioning, and Asset Integration

Opal features a natural-language-driven Theme Engine, letting builders describe visual styles (e.g., *sci-fi claymation cats*) to auto-generate custom color palettes and fonts.[15] Once generated and saved, these custom themes cannot be manually adjusted, though creators can switch between saved variations.[15]

Opal applications are stored as unified logical files in the user's Google Drive, featuring an automatic version history accessible via the top three-dot menu.[15] This panel lets builders preview and restore previous development states; however, restoring a historical version permanently deletes all iterations newer than the restored state.[15]

Builders can also upload reference photos to lock in a specific visual aesthetic, such as forcing a comic-strip app to maintain a consistent graphic style across different runs.[20] For advanced workflows, the system supports persistent document appends; each time a custom report app is run, it can append its new findings directly to the same Google Doc or Sheet, creating a continuous historical log.[20]

For complex logic, creators can transition from the standard interface to the Opal Advanced Editor at opal.google, which provides more granular workspace customization options.[21]

Gems from Google Labs: Orchestrating Autonomous Agentic Workflows

Gems from Google Labs—informally referred to as Super Gems—integrate the Opal app builder directly into the Gemini runtime environment, allowing creators to run multi-step automations without writing code.[2, 3]

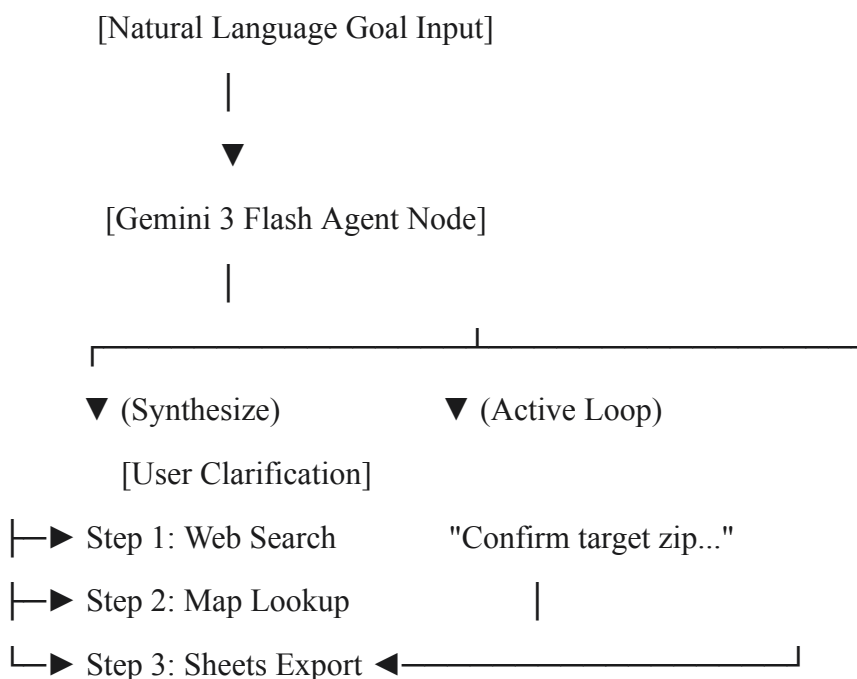
Operational Parameters and Constraints

Super Gems operate under a strict set of user requirements and beta limitations during their initial rollout [3]:

- **Account Sourcing:** Requires a personal Google Account with Google Labs and Opal clearances; school or corporate Workspace accounts are not supported in the initial beta phase.[3]
- **System and Device Restrictions:** Currently limited to desktop or laptop web browsers in English only.[3] The feature is not supported in the Gemini mobile app, Gemini in Google Messages, or the Gemini Chrome extension.[3]
- **Non-Convertibility:** Classic Gems and Super Gems are built on different logical engines; as a result, Classic Gems cannot be converted into Super Gems, nor can Super Gems be downscaled to Classic Gems.[3]
- **Data and Sharing Security:** Shared Super Gems reveal all instructions, prompts, and connected file titles to anyone with access.[10, 22] However, the actual contents of attached reference files are kept secure unless the builder explicitly grants separate sharing permissions for those underlying source files.[22]

The Plan-Then-Act Methodology

On February 24, 2026, Google upgraded Opal with an autonomous AI agent step powered by the Gemini 3 Flash model.[6, 23] This upgrade shifted Super Gems from simple, linear pipelines to dynamic, interactive applications.[6, 23] Under the "plan-then-act" model, when a user enters a complex goal, the agent plans the required tasks, self-selects built-in tools (such as Google Search, Google Maps, or Current Weather), and dynamically builds the application workflow on screen.[6, 15]



To prevent errors, the agent can pause the workflow to ask the user clarifying questions, resuming once the necessary details are provided.[6, 23] This agentic execution is secure, applying SynthID watermarks to all generated images and text to verify their AI origin.[6]

Visual Execution and the Developer Console

Super Gems organize complex prompts into sequentially visible steps, helping users identify errors or bottlenecked instructions easily.[1] This visual flow can be monitored in real time using the Developer Console, which displays the exact execution status and timing for each active step.[15]

Developer Console - Execution Log		
1. User Input [Completed]	0.0s	
2. Web Search [Completed - 3 sources found]	1.8s	
3. Gemini 3 Flash Generation [Executing...]	0.4s	
↳ Thinking Step: Analyzing search data...		
4. Google Sheets Output [Pending]	0.0s	

The console displays detailed inputs and outputs for every node, exposes the model's internal thinking steps, and logs real-time API and tool calls, allowing developers to isolate and test single steps during application debugging.[15]

Architectural Comparison of Custom AI Frameworks

This section contrasts Google's no-code app-building tools against professional code editors and other popular visual automation platforms, showing where each fits in the development lifecycle.

Google Opal vs. Cursor

This comparison highlights the differences between a visual no-code builder and a professional, AI-assisted code editor:

Architectural Metric	Google Opal No-Code Engine	Cursor AI-Assisted Code Editor
Primary Target Audience	Product managers, content creators, non-developers.[14]	Professional software developers and systems engineers.[14]
System Setup & Initialization	Instant browser-based setup using a Google login.[14]	Requires local installation, IDE configuration, and API keys.[14]
Code Access and Control	No direct access to code; limited to visual nodes.[14]	Full, unrestricted codebase access and environment control.[14]
Development Velocity	Fast prototyping; builds functional apps in minutes.[14, 19]	Slower initial setup; built for production-grade software.[14]
Best Target Use Cases	Internal workflow utilities and rapid prototyping.[14, 19]	Scalable, full-stack enterprise applications.[14, 19]
System Output Format	Visual workflows hosted entirely on Google servers.[14, 24]	Raw, deployable code files managed via version control.[14]
Core Pricing Model	Free of charge during its public beta phase.[14, 19]	Subscription model (typically \$20 per month for Pro).[14]

Comparative Matrix of No-Code App and Automation Builders

This matrix compares Opal to other visual app builders, visual coding assistants, and workflow automation platforms:

Feature / Metric	Google Opal [19]	Lovable [19]	Bubble [19]	n8n [19]	Zapier [19]	Replit [19]
Primary System Purpose	Visual AI app builder.[19]	Prompt-driven code generation.[19]	Visual full-stack app builder.[19]	Visual backend automation .[19]	Simple trigger-action workflows .[19]	Code-based software hosting.[19]
Core Architectural Strength	Workspace integration.[6, 19]	Rapid, clean front-end generation.[19]	Powerful drag-and-drop database.[19]	Highly flexible node-based logic.[19]	Massive third-party library.[19]	Full-stack cloud environments.[19]
User Skill Requirement	Non-technical.[19]	Semi-technical.[19]	Intermediate.[19]	Advanced.[19]	Non-technical.[19]	Developer.[19]
AI Native Design	Yes; built on Gemini.[14, 19]	Yes; uses LLMs.[19]	No; added as a feature.[19]	No; uses API plugins.[19]	No; added as a feature.[19]	No; features AI additions.[19]
Best Operational Use	Prototyping inside Workspace.[2, 19]	Exportable front-end web apps.[19]	Complete SaaS web applications.[19]	Technical backend automation s.[19]	Simple multi-app connections.[19]	Custom APIs and web applications .[19]
Primary Pricing Model	Free in public beta.[16, 19]	High tier subscriptions.[19]	Tiered visual pricing.[19]	Open-source or hosted tiers.[19]	Pay-per-execution model.[19]	Standard computing tiers.[19]

Subscription Economics and Developer Token Optimization

Integrating Gemini Gems, Super Gems, and Opal into a business workflow requires understanding the underlying subscription tiers, model execution limits, and API optimization strategies.

Consumer Subscription Framework

For personal and small-business accounts, Gemini's service tiers determine model access, monthly generation credits, and functional integration options:

Service Tier Plan	Pricing Level	Gem Operational Limits	Media Generation Allotments	Additional Cloud Benefits
Gemini Free	\$0 per month [25, 26]	Standard Gems built on Gemini 2.5 Flash with limited Pro access.[25, 26]	100 monthly credits for AI video and image creation.[25]	15 GB of shared Google account storage.[25]
Gemini 2x	\$4.99 per month [26]	Doubles the standard usage limits of the free plan.[26]	Standard generation limits.	Includes standard Google account benefits.[26]
Google AI Pro	\$19.99 per month [25, 26]	Gemini 2.5 Pro access; unlocks Gemini 3 models for US subscribers.[25]	1,000 monthly credits for Veo 3.1 video generation.[25]	Integration with Gmail/Docs; high-limit Code Assist.[25]
Google AI Ultra (Tier 1)	\$99.99 per month [26]	Unlocks Gemini 3.1 Pro and Gemini 2.5 Deep Think with 5x higher limits than Pro.[25, 26]	25,000 monthly credits for Veo 3.1 video generation.[25]	First access to Gemini Spark; \$100 Cloud credits.[25, 26]

Google AI Ultra (Tier 2)	\$199.99 per month [26]	Offers 20x higher usage limits compared to the AI Pro tier.[26]	High-volume media generation.	Premium Google Bundle features.[25, 26]
---------------------------------	-------------------------	---	-------------------------------	---

Enterprise Workspace Limit Enforcements

For corporate workspaces, Google has moved from early promotional access to strict limit enforcements to manage processing demand. These structural enforcements are scheduled to begin in mid-2026:

- **Workspace Studio Flows:** This service manages multi-step document automations.[27] Effective July 1, 2026, execution limits are capped at 100 flows per month for Business Starter, 400 for Business Standard, 2,000 for AI Expanded Access, and 10,000 for AI Ultra Access.[27]
- **AI Functions in Google Sheets:** Generative cell calculations will face limits starting July 15, 2026.[27] Business Starter has no access, while Standard, Plus, and Expanded plans are capped at 25,000 executions per user per month.[27]
- **Google Flow Credits:** This workspace orchestration service is restricted to 50 daily credits for standard tiers, 2,000 monthly credits for Expanded Access, and 25,000 monthly credits for Ultra Access.[26, 27]
- **Audio PDF Overviews:** High-quality voice synthesis summaries for documents are limited to 20 per day for Business Standard, 40 for Expanded, and 200 for Ultra.[27]

Developer API Optimization Strategy

For developers building outside the pre-configured Gemini web app, API integration requires balancing model capabilities with token consumption costs.[25, 28] Vertex AI and Google Cloud developers can use a structured optimization approach to manage operational costs [25, 29]:

- **Dynamic Response Routing:** Low-complexity tasks—such as summarization, extraction, and basic QA—should be dynamically routed to Gemini 3.1 Flash-Lite.[25, 29] This model is highly cost-effective, running at \$0.075 per million input tokens, which is significantly cheaper than Gemini 3.1 Pro.[25, 29] High-complexity tasks are escalated to Pro only when deep reasoning is required.[25]
- **Context Caching:** For workflows that repeatedly query static reference documents (such as corporate manuals, compliance policies, or large code libraries), developers should use context caching.[25, 28] This caches the document after the initial load, allowing subsequent queries to run at a heavily discounted rate (\$0.0125 to \$0.075 per million tokens depending on the model) alongside a storage fee of \$0.50 to \$1.80 per million tokens per hour.[25, 28, 29]
- **Asynchronous Batching:** For non-urgent processing tasks (such as overnight report generation or bulk document ingestion), developers can use batch mode.[25] This offers a

flat 50% discount on input and output token fees in exchange for up to 24 hours of processing latency.[25]

- **Context Window Pruning:** Long-running workflows accumulate history quickly.[25] To avoid the pricing cliff where costs double once input prompts exceed 200k tokens, applications should programmatically prune history to retain only essential context.[25, 28, 29]

Strategic Implications: The Future of Automation and Visual Synthesis

The development of Gemini Gems, Super Gems, and the Opal engine highlights several key trends in the custom AI software landscape.

Disintermediation of No-Code Middleware

The integration of visual app compiling directly into the Gemini interface challenges the business models of traditional RPA tools and third-party API aggregators.[2, 6, 19] Historically, custom AI automation required linking separate platforms—such as a front-end UI builder, an LLM API, a vector database, and cloud hosting—using middleware like Zapier or Make.com.[2, 19]

By unifying these tools into a single, cohesive workspace, Google allows users to go from a text prompt directly to a live, hosted web application with no technical setup.[2, 14, 15] This shift allows non-technical business units to quickly build, run, and share internal tools, reducing their reliance on central IT departments.[14, 19]

The Evolution of the Generative App Store

Opal's node-based approach, combined with direct link-sharing, establishes the foundation for a marketplace of custom, user-generated AI workflows.[2, 3] Because shared applications can be remixed with a single click, users can easily customize exist templates to suit their specific requirements.[3, 14, 15] This remixable gallery creates an ecosystem of shared tools, positioning Gemini as both an assistant and a platform for running custom, lightweight software applications.[2, 6, 14]

The Shift to Dynamic Visual Layouts

Historically, software development has relied on hard-coded user interfaces that remain unchanged regardless of the user's immediate task.[14, 19] Opal's auto-layout engine, however, compiles visual elements dynamically based on the generated data.[14, 15]

As natural language agents grow more sophisticated, this capabilities suggests a transition to dynamic, on-demand interface generation.[2, 14, 23] Instead of running pre-built software, future operating systems may dynamically generate custom visual interfaces to fit the immediate context, data structure, and goals of the user.[6, 14, 15]

-
1. Google Super Gems Update Builds Entire Workflows In 2 Minutes : r/AISEOInsider - Reddit,
https://www.reddit.com/r/AISEOInsider/comments/1t14436/google_super_gems_update_builds_entire_workflows/
 2. Google Super Gems — The Update That Turns Gemini Into an AI Factory - Julian Goldie, <https://juliangoldie.com/google-gemini-super-gems/>
 3. Create & manage AI mini-apps or custom workflows as Gems from Google Labs,
<https://support.google.com/gemini/answer/16802014?hl=en>
 4. I started using Gemini Gems, and now I get way more work done in less time,
<https://www.xda-developers.com/using-gemini-gems-get-more-work-done-in-less-time/>
 5. Google Gemini Gems - Hill Web Creations,
<https://www.hillwebcreations.com/google-gemini-gems/amp/>
 6. Google Launches AI Agent for Building Automated Workflows in Opal | MLQ News,
<https://mlq.ai/news/google-launches-ai-agent-for-building-automated-workflows-in-opal/>
 7. All About Google Gemini "Gems" [Thorough Comparison Edition] | やすとし - note,
<https://note.com/yasutoshiai/n/n9a8e86c6eccf?hl=en>
 8. Untitled, <https://services.newschool.edu/TDClient/32/Portal/KB/PrintArticle?ID=1364>
 9. Creating Custom Gems in Google.Ai - Chico State,
<https://support.csuchico.edu/TDClient/1984/Portal/KB/PrintArticle?ID=115135>
 10. Making Gems with Google Docs | Towards AI,
<https://towardsai.net/p/machine-learning/making-gems-with-google-docs>
 11. Tips for creating custom Gems - Gemini Apps Help,
<https://support.google.com/gemini/answer/15235603?hl=en>
 12. What are the best practices to write instructions for a Gemini Gem? : r/GeminiAI - Reddit,
https://www.reddit.com/r/GeminiAI/comments/1qdh8n/what_are_the_best_practices_to_write_instructions/
 13. Build custom experts with Gems - Google Gemini, <https://gemini.google/overview/gems/>
 14. Google Opal: Google's No-Code Tool for Building AI Apps | Codecademy,
<https://www.codecademy.com/article/google-opal-googles-no-code-tool>
 15. Overview | Opal | Google for Developers, <https://developers.google.com/opal/overview>
 16. Google Opal Availability - Where and How You Can Use It,
<https://opaltool.com/availability/>
 17. Google Labs introduces Opal for developing AI mini apps - InfoWorld,
<https://www.infoworld.com/article/4028370/google-labs-introduces-opal-for-developing-ai-mini-apps.html>
 18. Introducing Opal: describe, create, and share your AI mini-apps ...,
<https://developers.googleblog.com/introducing-opal/>

19. Google Opal Tutorial: A Complete Beginner's Guide to No-Code AI App Development, <https://www.datacamp.com/tutorial/google-opal-tutorial>
20. How to get started on Opal - YouTube, <https://www.youtube.com/watch?v=NWNNDvehBIU>
21. Build mini apps with Opal in the Gemini web app - Google Blog, <https://blog.google/innovation-and-ai/models-and-research/google-labs/mini-apps-opal-gemini-app-experiment/>
22. New Gem - Google Gemini, <https://gemini.google.com/gems/create?hl=en-US&pli=1>
23. Build dynamic agentic workflows in Opal - Google Blog, <https://blog.google/innovation-and-ai/models-and-research/google-labs/opal-agent/>
24. Opal - Google for Developers, <https://developers.google.com/opal>
25. Gemini Pricing in 2026 for Individuals, Orgs & Developers - Finout, <https://www.finout.io/blog/gemini-pricing-in-2026>
26. Google AI Pro & Ultra — get access to Gemini 3.1 Pro & more, <https://gemini.google/subscriptions/>
27. Compare Google AI expansion add-ons | Getting started, <https://knowledge.workspace.google.com/admin/getting-started/editions/compare-google-ai-expansion-add-ons>
28. Gemini Developer API pricing, <https://ai.google.dev/gemini-api/docs/pricing>
29. Agent Platform Pricing | Google Cloud, <https://cloud.google.com/gemini-enterprise-agent-platform/generative-ai/pricing>